MAKE MY MARRIAGE

API Design

REST API contract for private, public, and token-based Make My Marriage experiences

Project documentation | Version 1.0 | September 2026

Document Guide

This document is organized into the following sections:

1. API Conventions
2. Authentication and Authorization
3. Core Wedding APIs
4. Planning APIs
5. Guest and Public APIs
6. Media APIs
7. Permission Model
8. Validation, Errors and Rate Limits

1. API Conventions

- Base path: /api/v1
- Transport: HTTPS
- Format: JSON
- Private APIs require an authenticated session/JWT.
- Public and token-based APIs use explicit /public paths where appropriate.
- Server responses use stable error codes rather than frontend parsing of message strings.

2. Standard Responses

```
Success
{
  "success": true,
  "data": {}
}

List
{
  "success": true,
  "data": [],
  "meta": { "page": 1, "pageSize": 25, "total": 174, "totalPages": 7 }
}

Error
{
  "success": false,
  "error": { "code": "EVENT_NOT_FOUND", "message": "Event not found" }
}
```

3. HTTP Status Codes

Status	Meaning
200	OK
201	Created
204	No content
400	Invalid request
401	Not authenticated
403	Not authorized
404	Resource not found
409	Conflict
413	File too large
422	Validation error
429	Rate limited
500	Internal server error

4. Authentication

Method	Endpoint	Purpose
POST	/api/v1/auth/google	Authenticate/create user from verified Google identity.
GET	/api/v1/me	Return user, active membership and wedding summary.

```
POST /api/v1/auth/google
{ "idToken": "google-id-token" }

Existing member -> returns user + wedding
New user -> returns user + requiresWeddingSetup=true
```

5. Wedding APIs

Method	Endpoint	Permission
POST	/api/v1/weddings	Authenticated user without active wedding
GET	/api/v1/weddings/:weddingId	Admin/Member
PATCH	/api/v1/weddings/:weddingId	Admin; selected fields may allow Member later
DELETE	/api/v1/weddings/:weddingId	Admin
GET	/api/v1/weddings/:weddingId/dashboard	Admin/Member

```
POST /api/v1/weddings
{
  "brideName": "Priya",
  "groomName": "Ashu",
  "weddingDate": "2026-12-12",
  "city": "Jaipur",
  "state": "Rajasthan"
}

Server creates:
- wedding
- Admin membership
- unique slug
- Haldi, Mehndi, Sangeet, Wedding and Reception events
```

6. Dashboard API

```
GET /api/v1/weddings/:weddingId/dashboard

{
  "wedding": { "brideName": "Priya", "groomName": "Ashu", "daysRemaining": 42 },
  "events": { "total": 5, "upcoming": [] },
  "guests": { "total": 286, "attending": 198, "declined": 32, "pending": 56 },
  "tasks": { "total": 42, "completed": 34, "remaining": 8 },
  "expenses": { "total": 840000, "paid": 520000, "remaining": 320000, "currency": "INR" },
  "recentMedia": [],
  "recentActivity": []
}
```

7. Event APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/events	Admin/Member	List events
POST	/api/v1/weddings/:weddingId/events	Admin/Member	Create custom event
GET	/api/v1/events/:eventId	Admin/Member	Read event
PATCH	/api/v1/events/:eventId	Admin/Member	Update event
DELETE	/api/v1/events/:eventId	Admin/Member	Archive/delete custom event

8. Guest APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/guests	Admin/Member	List/search/filter guests
POST	/api/v1/weddings/:weddingId/guests	Admin/Member	Create guest
GET	/api/v1/guests/:guestId	Admin/Member	Read guest
PATCH	/api/v1/guests/:guestId	Admin/Member	Update guest
DELETE	/api/v1/guests/:guestId	Admin/Member	Archive/delete guest

9. Event Guest APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/events/:eventId/guests	Admin/Member	List assigned guests
POST	/api/v1/events/:eventId/guests	Admin/Member	Bulk assign guestIds
DELETE	/api/v1/events/:eventId/guests/:guestId	Admin/Member	Remove event assignment

10. Member APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/members	Admin	List members
POST	/api/v1/weddings/:weddingId/member-invites	Admin	Invite member by email
GET	/api/v1/member-invites/:token	Public	Read safe invite context
POST	/api/v1/member-invites/:token/accept	Authenticated invited user	Accept invite; email must match
DELETE	/api/v1/weddings/:weddingId/members/:memberId	Admin	Remove member

11. Task APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/tasks	Admin/Member	List/filter tasks
POST	/api/v1/weddings/:weddingId/tasks	Admin/Member	Create task
GET	/api/v1/tasks/:taskId	Admin/Member	Read task
PATCH	/api/v1/tasks/:taskId	Admin/Member	Update task
DELETE	/api/v1/tasks/:taskId	Admin/Member	Delete task

12. Expense APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/expenses	Admin/Member	List expenses

POST	/api/v1/weddings/:weddingId/expenses	Admin/Member	Create expense
GET	/api/v1/expenses/:expenseId	Admin/Member	Read expense
PATCH	/api/v1/expenses/:expenseId	Admin/Member	Update expense
DELETE	/api/v1/expenses/:expenseId	Admin/Member	Delete expense
GET	/api/v1/weddings/:weddingId/expenses/summary	Admin/Member	Totals by payment/category/event

13. Shopping APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/shopping-items	Admin/Member	List/filter items
POST	/api/v1/weddings/:weddingId/shopping-items	Admin/Member	Create item
PATCH	/api/v1/shopping-items/:itemId	Admin/Member	Update item
DELETE	/api/v1/shopping-items/:itemId	Admin/Member	Delete item

14. Vendor APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/vendors/search	Admin/Member	Search available vendors
GET	/api/v1/weddings/:weddingId/vendors	Admin/Member	List saved vendors
POST	/api/v1/weddings/:weddingId/vendors	Admin/Member	Save discovered/manual vendor
PATCH	/api/v1/wedding-vendors/:weddingVendorId	Admin/Member	Update wedding relationship
DELETE	/api/v1/wedding-vendors/:weddingVendorId	Admin/Member	Remove saved vendor
POST	/api/v1/events/:eventId/vendors	Admin/Member	Assign wedding vendor to event
DELETE	/api/v1/events/:eventId/vendors/:weddingVendorId	Admin/Member	Remove event assignment

15. Invitation APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/invitations	Admin/Member	List invitations
POST	/api/v1/weddings/:weddingId/invitations	Admin/Member	Create invitation
GET	/api/v1/invitations/:invitationId	Admin/Member	Read invitation
PATCH	/api/v1/invitations/:invitationId	Admin/Member	Update invitation
POST	/api/v1/invitations/:invitationId/publish	Admin/Member	Publish invitation

16. Livestream APIs

Method	Endpoint	Permission	Purpose

GET	/api/v1/weddings/:weddingId/livestreams	Admin/Member	List streams
POST	/api/v1/weddings/:weddingId/livestreams	Admin/Member	Create stream link
PATCH	/api/v1/livestreams/:livestreamId	Admin/Member	Update stream
DELETE	/api/v1/livestreams/:livestreamId	Admin/Member	Delete stream

17. RSVP APIs

Method	Endpoint	Permission	Purpose
POST	/api/v1/guests/:guestId/rsvp-link	Admin/Member	Create/rotate RSVP link
GET	/api/v1/public/rsvp/:token	Token	Return safe wedding/guest/event context
POST	/api/v1/public/rsvp/:token	Token	Submit event-level responses

```
POST /api/v1/public/rsvp/:token
{
  "responses": [
    { "eventId": "event1", "status": "ATTENDING" },
    { "eventId": "event2", "status": "NOT_ATTENDING" }
  ]
}
```

18. Public Wedding Website API

```
GET /api/v1/public/weddings/:slug

Returns only public-safe fields:
- couple names
- wedding date
- selected story/theme settings
- public event information
- public venue information
- gallery/livestream availability

Never returns members, guest list, expenses, tasks, internal vendor notes, or private
metadata.
```

19. Member Media Upload

```
1) POST /api/v1/weddings/:weddingId/media/upload-url
Request:
{
  "eventId": "...",
  "albumId": "...",
  "fileName": "IMG_1829.jpg",
  "mimeType": "image/jpeg",
  "fileSize": 7348921
}

Response:
{
  "mediaId": "...",
  "uploadUrl": "signed-s3-url",
  "expiresIn": 900
}

2) Browser uploads directly to S3

3) POST /api/v1/media/:mediaId/complete
```

20. Guest / QR Upload APIs

Method	Endpoint	Permission	Purpose
POST	/api/v1/weddings/:weddingId/upload-sessions	Admin/Member	Create guest upload session and QR link
GET	/api/v1/public/uploads/:token	Token	Return safe wedding name, allowed events and limits
POST	/api/v1/public/uploads/:token/files	Token	Validate file and return presigned S3 URL
POST	/api/v1/media/:mediaId/complete	Token/member context	Finalize media record after S3 upload

21. Album and Gallery APIs

Method	Endpoint	Permission	Purpose
GET	/api/v1/weddings/:weddingId/albums	Admin/Member	List albums
POST	/api/v1/weddings/:weddingId/albums	Admin/Member	Create album
GET	/api/v1/albums/:albumId	Admin/Member	Read album
PATCH	/api/v1/albums/:albumId	Admin/Member	Update album
DELETE	/api/v1/albums/:albumId	Admin/Member	Delete/archive album
GET	/api/v1/albums/:albumId/media	Admin/Member	Paginate media
POST	/api/v1/albums/:albumId/share-links	Admin/Member	Create private share link
DELETE	/api/v1/share-links/:shareLinkId	Admin/Member	Revoke share link
GET	/api/v1/public/gallery/:token	Token	View shared gallery

22. Activity and Demo APIs

Method	Endpoint	Permission	Purpose

GET	/api/v1/weddings/:weddingId/activity	Admin/Member	Recent wedding activity
GET	/api/v1/demo/wedding	Public	Read-only demo wedding payload

23. Permission Matrix

Capability	Admin	Member	Guest/Public
View dashboard	Yes	Yes	No
Edit wedding	Yes	Limited	No
Delete/archive wedding	Yes	No	No
Manage members	Yes	No	No
Manage events	Yes	Yes	No
Manage guests	Yes	Yes	No
Manage tasks	Yes	Yes	No
Manage expenses	Yes	Yes	No
Manage shopping	Yes	Yes	No
Manage vendors	Yes	Yes	No
Upload photos	Yes	Yes	With token
Create gallery share link	Yes	Yes	No
RSVP	-	-	With token
View public wedding website	Yes	Yes	Yes
View shared gallery	Yes	Yes	With token

24. Pagination, Filtering and Sorting

```
Pagination: ?page=1&pageSize=25   (max 100)
Sorting:    ?sortBy=createdAt&sortOrder=desc
Search:     ?search=sharma
Filters:    resource-specific, e.g. ?side=BRIDE&status=ACTIVE
```

Sort fields must be allow-listed. Large activity and media collections can move to cursor pagination later.

25. Validation Rules

- Use a shared server-side schema validation library such as Zod.
- Validate ObjectIds, enum values, dates, emails, slugs, amounts, paidAmount <= amount, MIME types and file sizes.
- Frontend validation improves UX but never replaces backend validation.

26. Stable Error Codes

```
UNAUTHENTICATED
FORBIDDEN
WEDDING_NOT_FOUND
EVENT_NOT_FOUND
GUEST_NOT_FOUND
VENDOR_NOT_FOUND
MEMBER_NOT_FOUND
INVALID_INVITE
INVITE_EXPIRED
INVALID_RSVP_TOKEN
INVALID_UPLOAD_TOKEN
UPLOAD_LIMIT_EXCEEDED
FILE_TYPE_NOT_ALLOWED
FILE_TOO_LARGE
SLUG_ALREADY_EXISTS
RESOURCE_CONFLICT
VALIDATION_ERROR
```

27. Authorization Pipeline

```
Authenticated endpoint:
Authenticate -> Resolve resource -> Resolve weddingId -> Check ACTIVE membership -> Check
role -> Validate input -> Execute -> Audit

Token endpoint:
Rate limit -> Hash token -> Resolve active token/session -> Validate expiry/scope -> Return
safe data or execute bounded action
```

28. Rate Limiting

- Apply stricter limits to /auth and all /public token endpoints.
- Guest upload endpoints should use per-IP and per-token controls plus upload count/size limits.
- Public website endpoints may use CDN/cache controls but should still protect origin APIs.

29. Recommended Implementation Order

9. Auth + user + wedding + membership.
10. Events + guests + event assignments.
11. Tasks + expenses + shopping.
12. Member invites + activity.
13. RSVP + invitation + public wedding website.
14. Wedding vendors + event vendors.
15. Albums + S3 media upload.
16. QR guest upload + share links.
17. Livestream + demo + dashboard aggregation tuning.